```
  parents[v] ← null (" v ∈
graph)                                                  #5
  distances[start] ←
0                                                             #6
  while not queue.empty()
do                                                          #7
     v ←
queue.top()
      #8
     if isGoal(v)
then                                                              #9
       return v, parents
     else
       for e in graph.adjacencyList[v]
do                                        #10
         u ← e.dest
         if distances[u] < distances[v] + e.weight
then                          #11
           distances[u] ← e.weight +
distances[v]                                  #12
           parents[u] ← v
           queue.update(u, distances[u])
  return null,
parents
#13
```

#1 Method `dijkstra` takes a graph, a starting point vertex, and a predicate (`isGoal`) that takes a vertex and returns `true` if the goal of the search is reached. Method `dijkstra` returns a pair with the goal vertex (if reached), and a dictionary encoding the shortest paths.

#2 Initialize a priority queue. This concretely could be, for instance, a heap.

#3 Add the starting point to the priority queue, using its distance (0) as its priority, so that it will be extracted on